

LAB Assignment: Music Generation with RNNs
Course Name and instructor: GAI, Prof G C Nandi
Date of announcement: 08-04-2026
Date of submission: 16-04-2026 by mid night (12:00 am)
Total Marks : 50

In this assignment, you will implement three different recurrent architectures (vanilla RNN, GRU, and LSTM) to generate musical sequences. This will demonstrate how each architecture handles temporal dependencies in musical data. This assignment is implementable in your laptop with CPU.

Dataset

The Nottingham Music Database (folk tunes in ABC notation)

- Dataset URL: <https://github.com/jukedeck/nottingham-dataset>
- The dataset contains over 1,000 folk tunes in a simple, character-based format

Part 1: Data Preprocessing

You need to:

1. Convert ABC notation into a sequence of tokens (notes, durations, and rests)
2. Create training sequences of fixed length (e.g., 64 tokens)
3. Implement one-hot encoding for the tokens

Part 2: Model Implementation

You should implement three models with identical architectures except for the recurrent layer:

1. Vanilla RNN model
2. GRU model
3. LSTM model

Each model should have:

- An embedding layer
- The recurrent layer (RNN/GRU/LSTM)
- A dense output layer with softmax activation
-

Part 3: Training and Evaluation

1. Train each model with the same hyperparameters
2. Plot training and validation loss curves
3. Generate music samples using each model with varying temperature settings
4. Compare the coherence of the generated music over different sequence lengths

Part 4: Analysis

You need to analyze:

1. How well each model maintains musical structure over time
2. The ability of each model to learn long-term dependencies (e.g., repeating themes)
3. Training efficiency (time and convergence rate)
4. Model size and computational requirements

Suggestive codes for implementations (you may generate your own):

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Dense, SimpleRNN, GRU, LSTM
from tensorflow.keras.callbacks import ModelCheckpoint
import matplotlib.pyplot as plt
import os
import re

# Data loading function
def load_abc_data(data_dir):
    """Load all ABC notation files from directory"""
    all_text = ""
    for file in os.listdir(data_dir):
        if file.endswith(".abc"):
            with open(os.path.join(data_dir, file), 'r') as f:
                all_text += f.read() + "\n"
    return all_text

# Data preprocessing
def preprocess_abc_data(text):
    """Clean and tokenize ABC notation"""
    # Remove headers and keep only tune bodies
    tunes = re.findall(r'X:.*?K:.*?\n(.*?)(?=X:|$)', text, re.DOTALL)
    processed_tunes = []
    for tune in tunes:
        # Remove comments and whitespace
        cleaned = re.sub(r'%.*$', '', tune, flags=re.MULTILINE)
        cleaned = re.sub(r'\s+', ' ', cleaned).strip()
```

```

        if cleaned:
            processed_tunes.append(cleaned)

    return processed_tunes

# Create training sequences
def create_sequences(texts, seq_length=64):
    """Create input-output pairs for training"""
    chars = sorted(list(set(''.join(texts))))
    char_to_idx = {c: i for i, c in enumerate(chars)}
    idx_to_char = {i: c for i, c in enumerate(chars)}

    sequences = []
    next_chars = []

    for text in texts:
        for i in range(0, len(text) - seq_length):
            sequences.append(text[i:i + seq_length])
            next_chars.append(text[i + seq_length])

    X = np.zeros((len(sequences), seq_length), dtype=np.int32)
    y = np.zeros((len(sequences), len(chars)), dtype=np.bool)

    for i, sequence in enumerate(sequences):
        for t, char in enumerate(sequence):
            X[i, t] = char_to_idx[char]
            y[i, char_to_idx[next_chars[i]]] = 1

    return X, y, char_to_idx, idx_to_char

# Model building function
def build_model(model_type, vocab_size, seq_length, embedding_dim=128, rnn_units=256):
    """Build RNN/GRU/LSTM model for music generation"""
    model = Sequential()
    model.add(Embedding(vocab_size, embedding_dim, input_length=seq_length))

    if model_type == 'rnn':
        model.add(SimpleRNN(rnn_units, return_sequences=False))
    elif model_type == 'gru':
        model.add(GRU(rnn_units, return_sequences=False))
    elif model_type == 'lstm':
        model.add(LSTM(rnn_units, return_sequences=False))

    model.add(Dense(vocab_size, activation='softmax'))

    model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
    return model

```

```

# Music generation function
def generate_music(model, char_to_idx, idx_to_char, seed_text, num_chars=500, temperature=1.0):
    """Generate music using the trained model"""
    generated = seed_text

    for _ in range(num_chars):
        x_pred = np.zeros((1, len(seed_text)))
        for t, char in enumerate(seed_text):
            x_pred[0, t] = char_to_idx[char]

        preds = model.predict(x_pred, verbose=0)[0]

        # Apply temperature scaling
        preds = np.log(preds) / temperature
        exp_preds = np.exp(preds)
        preds = exp_preds / np.sum(exp_preds)

        # Sample next character
        next_index = np.random.choice(len(preds), p=preds)
        next_char = idx_to_char[next_index]

        generated += next_char
        seed_text = seed_text[1:] + next_char

    return generated

# Main training and evaluation code here
# You need to implement this part

```

Deliverables

1. Jupyter notebook with implementation and analysis
2. Generated music samples from each model type (as MIDI files)
3. A 2-3 page report comparing the three architectures and their performance in music generation.